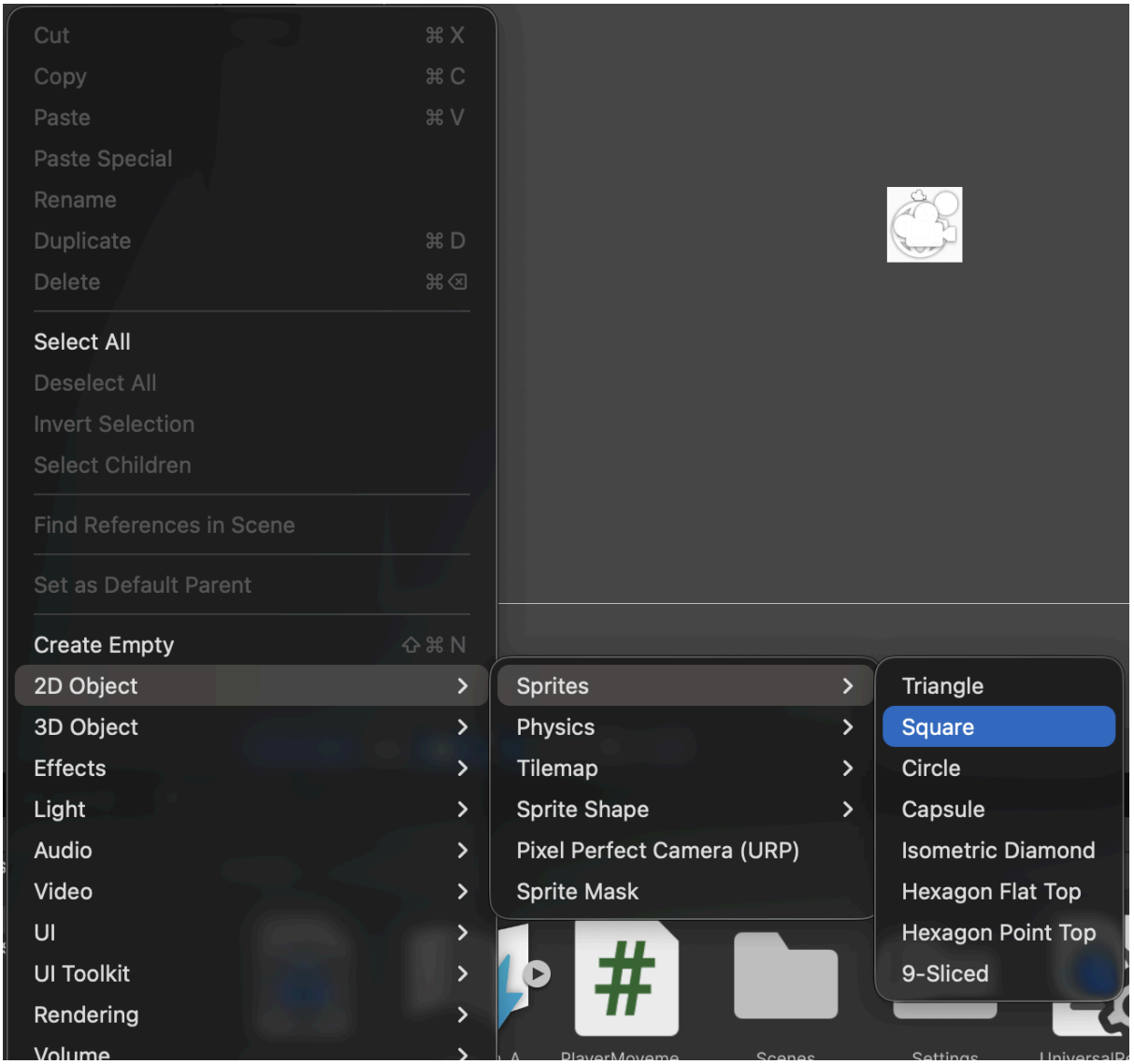
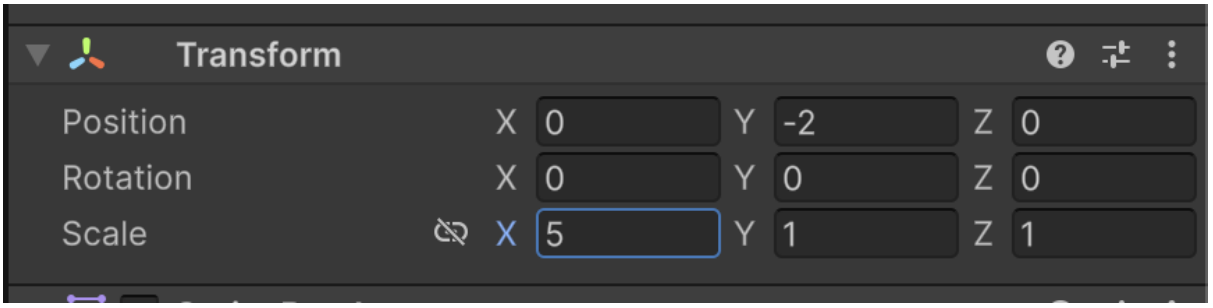
Hello everyone, and welcome back to another Unity tutorial, this time on advanced player movement! In this tutorial, we will take our player from tutorial no. 2 and make them experience gravity, jump, and let the camera follow them around!

If we go back to our project from tutorial 2, we left the player able to move from side to side, but it did not experience gravity and was not able to move up and down. Unless you're making a Pong knockoff, this won't be enough. First, we're going to want to let the player experience gravity again. Go to the Rigidbody component of your Player object and find the "Gravity Scale" setting. We set it to 0 in our last tutorial, but we're going to set it to 1 again. If you press play now, you will see that the player object quickly falls off the screen. This is because we haven't made anything to hold it up yet. Go to the Hierarchy tab (left of the screen) and right click. Navigate to 2D Object > Sprites > Square, like so:



Select the "Square" option and another white square should appear in your game window. (it may be overlapped by your player object so it might not immediately be visible; it will clearly appear as "Square" in your Hierarchy menu. Select the square in the Hierarchy menu by left-clicking it - then go to the Inspector tab (right of the screen) and look at the Transform component of the square. This component controls the position, rotation and scale (size) of

the object along all 3 axes. Set the Y component of the Position to be -2 and the X component of the Scale to be 5, like so:



This will have moved the square object 2 units down relative to the player. If you press the Play button now, unfortunately the player will still fall directly through the square. We haven't done anything wrong; the game just hasn't been told to recognise that these are solid objects. To do that, we need to add another component to both the square and the player. For both the square and the player, left click on them in the Hierarchy, go to the Add Component button in their Inspector tab (bottom of the Inspector tab - you may need to scroll down), and in the search bar that pops up, write "Box Collider 2D", and select the resulting component. Once you have a Box Collider on both objects, you can hit play.

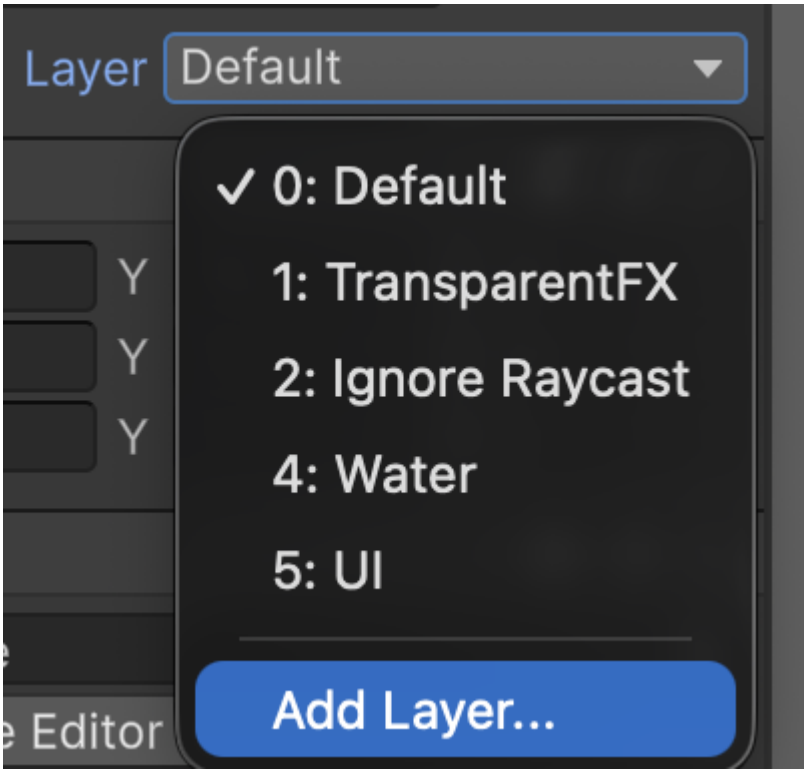
If everything was done correctly, you will now see the player fall onto the square platform and stop. You can move around as normal, but if you fall off the sides of the platform, you will keep falling off the screen. Now, if you go slowly to the very edge of the platform, you will notice that you player begins to spin as it falls off the edge. This is a sign that the player may go into a spin if you hit some other object at a weird angle. You may want this in your game, but if you do not, go to your Rigidbody 2D component of your player and select the checkbox in Constraints > Freeze Rotation > Z.

Let's allow our player to jump. Again, if you downloaded the code-free package recommended by my previous tutorial, you probably already have the ability to jump installed. But if you want to code your own stuff, continue on. Open up your "PlayerMovement2D" script, and make a new line under the line "private float moveSpeed = 10f;" (If nothing's been changed, this would be line 14) and write "private float jumpHeight = 5f;" on this new line. Then, go to the line "player.linearVelocity = new Vector2(Input.GetAxis("Horizontal") * moveSpeed, player.linearVelocity.y);" (If nothing's been changed, this would now be Line 25), make four new lines underneath, and write in the following:

```
if (Input.GetKeyDown(KeyCode.Space))
{ // this if statement checks if the player pressed space and if so, lets them jump.
    player.linearVelocity = new Vector2(player.linearVelocity.x, jumpHeight);
}
```

This code contains an "if statement" a form of code that's common in all coding languages. Basically, it means that IF a specific condition (in this case, the key "Space" being pressed) is true, then perform the action enclosed by the brackets.

If we go back to Unity, wait for the new code to compile, and then hit Play, we can now press Space to jump! However, the astute coders may have noticed that we can jump unlimited times in mid-air. This is an issue, because it basically allows for flight. There's a fairly easy solution, however, which is to select the square platform in the Inspector tab, find the "Layer" box (top right of Inspector tab), select the box and hit "Add Layer", like so:



Now in the resulting menu that pops up, type "Ground" into the first empty "user layer" box that you see in the list. Then, select the square platform in the Hierarchy again, and navigate back to the "Layer" Menu shown above. The layer "Ground" should now have appeared, alongside a number. You will need to remember this number, because it is how the layer is referenced in your code. Select the Ground layer. Now, go back to your Player movement script.

1. Go to line 15, make a new line under it, and write "private bool is_on_ground = false;"
2. Go to line 27 (your if statement) and change the line to:
   if (Input.GetKeyDown(KeyCode.Space) && is_on_ground)
3. Go to line 30 (or wherever the void Update method has its last bracket, make a new line below, and paste the following code into it:

```
void OnCollisionEnter2D(Collision2D col)
{
    if (col.gameObject.layer == 3)
    {
        is_on_ground = true;
    }
}
void OnCollisionExit2D(Collision2D col)
{
```

```
    if (col.gameObject.layer == 3)
    {
        is_on_ground = false;
    }
}
```

This code essentially creates a boolean (a variable that is either true or false), which is set to true when the player touches the ground and false when they leave the ground, and change the if statement so that it only lets the player jump when they're currently on the ground.

NOTE: The "3" in both of these if statements may or may not be the layer number that you have inputted. Check which number is next to your "Ground" layer in the layer menu and use that number in place of 3.

Once you hit play, your player will not be able to jump in mid-air anymore - to the dismay of speedrunners and playtesters worldwide.

Finally, we're onto camera movement. To get the camera to follow the player around, simply go to the Hierarchy tab, and select the "Main Camera" object, and drag and drop it onto your "Player" object. It should appear indented underneath the Player in the Hierarchy tab. This means you have just made the Camera a 'child' of the player, and as such its position, rotation and scale are now measured relative to the player. This also has the handy effect of making the camera follow the player.

If you hit play now, your player will appear to remain still whilst your platform moves around. This is simply due to the featureless background. The player is still the one that is moving, but due to the camera moving with it, it seems like it's the platform that's moving. Feel free to create a couple more platforms and scatter them around the world, and jump between them to make a fun platformer game!